A. K-Color Union

10 s, 256 megabytes

The city of Chromatica has N landmarks connected by $N-1$ bidirectional roads forming a tree structure. Each landmark is painted with one of K distinct colors (numbered 1 to K).

The city's tourism board wants to promote "rainbow tours" — tours that visit all K colors at least once. A tour is defined as starting at one landmark and ending at another, following the unique path between them.

To design their marketing campaign, they need to know: how many distinct pairs of landmarks can form a rainbow tour?

Input

The first line contains a single integer T ($1 \le T \le 10$), the number of test cases. The description of T test cases follows.

Each test case is described as follows:

- The first line contains two integers N and K ($1 \le N \le 10^5$, $1 \le K \le 10$).
- The second line contains N integers $c_1, c_2, \ldots, c_N$ ($1 \le c_i \le K$), where c_i is the color of the i-th landmark.
- Each of the next $N-1$ lines contains two integers u and v ($1 \le u, v \le N$, $u \ne v$), describing an undirected road between landmark u and landmark v.

It is guaranteed that the roads form a tree, and the sum of N over all test cases does not exceed 10^5.

Output

For each test case, output a single integer — the number of unordered pairs (u, v) such that the path from u to v contains at least one landmark of each of the K colors.

input

```
2
4 3
1 2 3 2
1 2
2 3
3 4
5 3
1 1 2 3 2
1 2
2 3
3 4
4 5
```

output

```
2
4
```

B. Nonmetal Alchemist

2 seconds, 256 megabytes

In the land of Amestris, alchemists are known for bending matter to their will, shaping stone, fire, and metal in the name of war or progress. But among them stands one who chose a different path — Rutherford, the Nonmetal Alchemist. Unlike others, he turned his back on combat and committed his life to the pursuit of knowledge.

Lately, usage of transmutation across Amestris has grown in intensity, leading to the appearance of never-before-seen elements at an alarming rate! These elements possess unknown properties, and Rutherford believes that their electron configurations, more specifically, the number of electrons in the final shell of their atoms, hold the key to understanding them.

Every element in the universe is made of atoms, and each atom in its neutral state has a specific number of electrons equal to its atomic number Z. These electrons orbit the nucleus, and each electron is uniquely described by four integers called **quantum numbers**.

- The principal quantum number n: Each value of n uniquely describes a shell, where $n \ge 1$
- The azimuthal quantum number ℓ: Each (n, ℓ) pair uniquely describes a subshell, where $0 \le \ell < n$
- The magnetic quantum number m: Each (n, ℓ, m) triplet uniquely describes an orbital, where $-\ell \le m \le \ell$
- The spin quantum number s: Each (n, ℓ, m, s) tuple uniquely describes an electron, where $1 \le s \le S$

The number of electrons that can fill in a subshell (n, ℓ) is determined by the **number of valid** (n, ℓ, m, s) **combinations** (each representing an electron). For example, if $S = 2$, the subshell $(2, 1)$ has a capacity of 6 electrons. The quantum numbers for those 6 electrons are $(2, 1, -1, 1)$, $(2, 1, -1, 2)$, $(2, 1, 0, 1)$, $(2, 1, 0, 2)$, $(2, 1, 1, 1)$, $(2, 1, 1, 2)$.

For any atom, electrons fill subshells in **increasing order of energy**, following the Aufbau principle. The energy of a subshell is primarily determined by $n + \ell$, and in case of ties, the subshell with the smaller n is filled first. According to this rule, the first 10 subshells filled are: $(1, 0)$, $(2, 0)$, $(2, 1)$, $(3, 0)$, $(3, 1)$, $(4, 0)$, $(3, 2)$, $(4, 1)$, $(5, 0)$, $(4, 2)$. The **final shell** of an atom is the largest value of n that has a non-zero number of electrons.

For $S = 2$, the electrons of Germanium, an atom with atomic number 32, enter the subshells in the following order:

- First, 2 electrons enter the subshell $(1, 0)$, completely filling it.
- Then, 2 electrons enter the subshell $(2, 0)$, completely filling it.
- Then, 6 electrons enter the subshell $(2, 1)$, completely filling it.
- Then, 2 electrons enter the subshell $(3, 0)$, completely filling it.
- Then, 6 electrons enter the subshell $(3, 1)$, completely filling it.
- Then, 2 electrons enter the subshell $(4, 0)$, completely filling it.
- Then, 10 electrons enter the subshell $(3, 2)$, completely filling it.
- Finally, the remaining 2 electrons enter the subshell $(4, 1)$.

The electron configuration of Germanium can be written in shorthand as $(1, 0)^2 (2, 0)^2 (2, 1)^6 (3, 0)^2 (3, 1)^6 (3, 2)^{10} (4, 0)^2 (4, 1)^2$, where $(n, \ell)^k$ means that k electrons are placed in the subshell (n, ℓ). So, the final shell for this atom is shell 4, and the number of electrons on the final shell is $2 + 2 = 4$.

You are Bohr, Rutherford's student — curious, determined, and eager to assist your mentor in his mission. Help Rutherford calculate **the number of electrons in the final shell** for any atom, given its atomic number Z and S, the number of possible spin values for electrons in this universe.

Input

The first line contains a single integer t ($1 \le t \le 10^5$) — the number of test cases.

The first line of each test case contains two space-separated integers q ($1 \le q \le 10^5$) and S ($1 \le S \le 10^5$) — the number of atoms and the number of possible spin values for electrons in this universe.

The second line of each test case contains q space-separated integers $Z_1, Z_2, \ldots, Z_q$ ($1 \le Z_i \le 10^{12}$) — where Z_i is the atomic number of the i-th atom.

It is guaranteed that the sum of q over all test cases doesn't exceed 10^6.

Output

For each test case, output q space-separated integers in a line — the i-th of which should be the number of electrons in the final shell for the i-th given atom in the given universe.

Output tokens may be separated by arbitrary whitespace (spaces, tabs, or newlines). Trailing whitespace at the end of lines or at the end of the output will not affect the verdict.

input
4 1 2 32 10 2 9 10 11 12 17 18 19 20 35 36 1 7 35 2 2 1 1000000000000

output
4 7 8 1 2 7 8 1 2 7 8 28 1 2

The first test case is explained in the statement.

In the third case, the electron configuration is $(1,0)^7(2,0)^7(2,1)^{21}$. So, in a universe with 7 electron spins, Bromine is a noble gas.

C. Self Citation

3 s, 256 megabytes

Professor Aubergine cares a lot about his citation count. He wants to increase his total number of citations as much as possible.

He plans to publish exactly n papers over the next k months. In the i-th month, he can publish at most a_i papers due to submission limits.

Every paper he publishes in a month self cites all of his papers published in all previous months. However, papers published in the same month cannot cite each other.

Let b_i be the number of papers he publishes in month i. Your task is to find a sequence $b_1, b_2, \ldots, b_k$ that gives the maximum possible number of total citations. The sequence must satisfy $0 \le b_i \le a_i$ for all i, and the total sum of all b_i must be exactly n.

Input

The first line contains a single integer t ($1 \le t \le 10^3$) — the number of test cases. Then the description of the test cases follows.

The first line of each test case contains two space-separated integers n and k ($1 \le n \le 2 \cdot 10^{15}$, $1 \le k \le 2 \cdot 10^6$) — the total number of papers and the number of months.

The second line of each test case contains k space-separated integers $a_1, a_2, \ldots, a_k$ ($0 \le a_i \le 10^9$, $n \le a_1 + a_2 + \cdots + a_k$) — the maximum number of papers he can publish in each month.

It is guaranteed that the sum of k over all test cases does not exceed $2 \cdot 10^6$.

Output

For each test case, print k space-separated integers $b_1, b_2, \ldots, b_k$ — the optimal number of papers to publish each month.

If there are multiple sequences that give the same maximum number of citations, you may print any of them.

input
2 10 3 3 4 5 12 3 2 4 10

output
3 3 4 2 4 6

In the first test case, the submission limits for the 3 months are $a = [3, 4, 5]$ and we need to choose a sequence that sums up to $n = 10$.

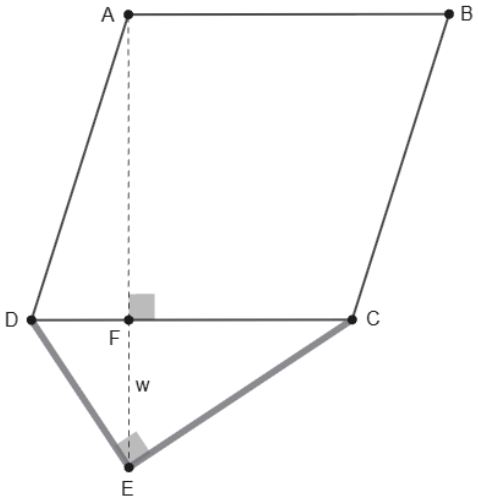
If we choose $b = [3, 4, 3]$, the citations are calculated as follows: Number of self citations in the second month = $4 \times 3 = 12$ Number of self citations in the third month = $3 \times (3 + 4) = 21$ Total citations = $12 + 21 = 33$.

Alternatively, $b = [3, 3, 4]$ is also a valid answer.

D. Floatovoltaics

2 seconds, 256 megabytes

To address the growing demand for renewable energy, the Department of Efficiency has commissioned a massive floating solar farm to be deployed on the tranquil surface of Kaptai Lake. You have been appointed as the Chief Geometry Engineer for this ambitious project.



The array of solar panels must be constructed in the shape of a perfect rhombus, denoted as $ABCD$, where $\angle ADC$ is acute. To transmit the generated electricity to the national grid, the array must be connected to a high-voltage transmission hub located at point E.

To establish this connection, two heavy-duty submarine cables will be laid from the hub E to the closest corners of the solar array, C and D. To perfectly balance the hydrodynamic tension caused by the lake's shifting currents, these cables, CE and DE, must converge at the hub at a perfect right angle (i.e., $\angle CED = 90^\circ$).

The solar array $ABCD$ and the hub E must be situated on strictly opposite sides of the line CD. Furthermore, strict engineering protocols require that the imaginary line connecting point A directly to point E must be strictly perpendicular to the edge CD. Line AE intersects the edge CD at point F.

The distance from the edge of the array to the transmission hub, which is the length of segment FE, is immutably fixed at w meters due to the depth of the coastal drop-off. However, as the Chief Geometry Engineer, you have the flexibility to adjust the dimensions and position of the array (for instance, by changing the lengths of DF and CF) to minimize the total cost of the project.

The construction costs are calculated as follows:

- **Panel Cost:** Assembling the floating solar panels costs x taka per unit area.
- **Cabling Cost:** Laying the reinforced submarine cables costs y taka per unit length.

The total project cost is the sum of the **Panel Cost** and the **Cabling Cost**:

$$\textbf{Total Project Cost} = x \cdot [ABCD] + y \cdot (CE + DE)$$

Here, $[ABCD]$ denotes the area of the rhombus $ABCD$.

Given w, x, and y for several proposed deployment sites across the lake, find the minimum possible total project cost.

Input
The first line contains a single integer t ($1 \le t \le 10^5$) — the number of test cases.

Each test case consists of a single line containing three integers w, x, and y ($1 \le w, x, y \le 10^6$) — the fixed coastal drop-off distance FE, the panel cost multiplier, and the cabling cost multiplier, respectively.

Output
For each test case, output a single real number — the minimum possible total project cost in taka.

Your answer will be considered correct if its absolute or relative error does not exceed 10^{-6}. Formally, let your answer be a, and the jury's answer be b. Your answer is accepted if $\frac{|a-b|}{\max(1,|b|)} \le 10^{-6}$.

input
3 1 1 1 5 10 2 100 50 50

output
6.20016399 861.43628993 1679544.09387086

E. Min-XOR MST

1 second, 256 megabytes

The King of XORia wants to pave a set of dirt roads to connect all N cities in his kingdom. There are M bidirectional dirt roads available, and paving a specific road costs W gold coins. The goal is to pave a subset of these roads so that people can travel between any two cities, while keeping the total cost as small as possible.

The King's wizard has offered him a magical spell with a power of X. The King can choose exactly one road to cast this spell on. If the spell is cast on a road that originally costs W coins, its paving cost changes to $(W \oplus X)$ coins, where $\oplus$ represents the bitwise XOR operation.

If the King optimally chooses which road to cast the spell on, what is the minimum possible total cost to connect all the cities?

Input
The first line contains a single integer T ($1 \le T \le 10^4$) — the number of test cases.

The first line of each test case contains three integers N, M, and X ($2 \le N \le 2 \cdot 10^5$, $N - 1 \le M \le 2 \cdot 10^5$, $1 \le X \le 10^9$) — the number of cities, the number of roads, and the spell power, respectively. Each of the next M lines contains three integers u, v, and W ($1 \le u, v \le N$, $1 \le W \le 10^9$), meaning there is a bidirectional dirt road between city u and city v that costs W gold coins to pave.

It is guaranteed that it is possible to travel between any pair of cities using the initial set of dirt roads. Additionally, the sum of N over all test cases does not exceed $2 \cdot 10^5$, and the sum of M over all test cases does not exceed $2 \cdot 10^5$.

Output
For each test case, print a single integer — the minimum total gold cost to pave a connected network of roads spanning all cities after performing the magical operation on exactly one road.

input
1 4 5 2 1 2 5 2 3 6 3 4 7 4 1 8 1 3 9

output
16

F. Fool's Gold

3 seconds, 256 megabytes

In the Kingdom of Chemistry, every citizen has a unique **elemental ID** ranging from 1 to n. King Pyrite, whose elemental ID is n, rules over the kingdom with an aura of false purity.

To solidify his rule, the king seeks to form an elite council known as **The Noble Catalysts**. These individuals will be publicly regarded as the most brilliant minds in the kingdom, yet their true purpose is not to provide wisdom but to accelerate the king's plans without resistance. Like chemical catalysts, they thrive under King Pyrite's influence, ensuring that his decrees are swiftly accepted by the people without question.

The king has two strict criteria for selecting his advisors:

- **Loyalty** — An advisor must always agree with the king's decisions.
- **Credibility** — The citizens must never doubt the intelligence or capabilities of the chosen advisors.

The kingdom's Royal Alchemist has prophesied that for two citizens with elemental IDs x and y where $x < y$, the person with ID x will always agree with the person with ID y if the following condition holds:

$$\gcd(x, y) + \operatorname{lcm}(x, y) = x + y$$

Here $\gcd(x, y)$ denotes the greatest common divisor of x and y, and $\operatorname{lcm}(x, y)$ denotes the least common multiple of x and y.

However, there is a catch. To maintain the illusion of a flawless selection, King Pyrite will only invite citizens whose elemental ID is a perfect square, as these individuals are perceived as the "purest" in the kingdom.

Given the value of n, determine how many citizens can be selected as advisors by King Pyrite such that all of the following **conditions** hold:

- They will always agree with the king.
- x is a perfect square.
- $x < n$ (the king himself cannot be an advisor).

Input
The first line of the input contains a single integer t ($1 \le t \le 10^5$) — the number of test cases.

Each test case consists of a single line containing a single integer n ($2 \le n \le 10^9$) — the number of citizens.

Output
For each test case, output a single integer in a line — the number of citizens that can be selected as advisors.

input
4 10 16 25 100000000

output
1 2 1 24

In the second test case, the citizens with elemental ID 1 and 4 can be advisors.

G. Dynamic Ranklist

1.5 seconds, 512 megabytes

The organizers of a programming training camp are running a long practice contest. Initially, participant i belongs to a team containing only i for every $1 \le i \le N$. As the camp progresses, coaches may merge teams, and the live scoreboard must immediately reflect the new team standings.

There is one complication: verdict logs arrive from multiple judging servers. The events are processed in the order they are received, but a submission event contains its actual contest timestamp. Therefore, an accepted submission reported later may have happened earlier than submissions already processed.

You are given all events received by the scoreboard system. During the contest, two kinds of events may occur:

- A submission verdict arrives.
- Two existing teams merge into one team.

Submission verdicts are processed in the order they appear in the input. However, each submission also has a timestamp t, which is the actual contest time when the submission was made.

Whenever a team receives a new verdict or two teams merge, the team's score is recalculated using all submissions currently belonging to that team.

Scoring Rules

For each team:

- The solve count is the number of distinct problems for which the team has at least one accepted verdict.
- The total penalty is the sum of the penalties of all solved problems.

For one solved problem:

- A team may have multiple accepted submissions for the same problem. Let T be the minimum timestamp among all accepted submissions by the team for that problem.
- Let W be the number of wrong-answer submissions by the team for that problem with timestamp strictly less than T.
- The problem penalty is $T + 20 \cdot W$.

Unsolved problems contribute 0 penalty.

Ranking Rules

A team ranks strictly better than another team if, in order:

1. It has a larger solve count.
2. If solve counts are equal, it has a smaller total penalty.
3. If both are equal, it has a smaller minimum participant ID among its members.

The rank of a team is $1+$ the number of currently existing teams that rank strictly better than it.

Input

The first line contains one integer T ($1 \le T \le 100000$), the number of test cases.

For each test case, the first line contains two integers N and Q ($1 \le N, Q \le 100000$), where N is the number of participants and Q is the number of events.

Each of the next Q lines describes one event.

For a type 1 event, the format is `1 t u p v`. Here:

- t is the chronological submission timestamp.
- u is the participant who made the submission. The submission belongs to the team containing u at that moment.
- p is the problem ID.
- v is the verdict: 0 means wrong answer, and 1 means accepted.

For a type 2 event, the format is `2 u v`. The current teams containing participants u and v merge into a single team. If they are already in the same team, the operation does nothing.

It is guaranteed that:

- $1 \le t \le 10^9$
- $1 \le u, v \le N$
- $1 \le p \le 100000$
- Within one test case, all submission timestamps are distinct.
- In one input file, the sum of all N values is at most 100000.
- In one input file, the sum of all Q values is at most 100000.

Output

For each test case, output exactly $Q + N$ lines.

After each event, print three integers in the format `Rank Solves Penalty`.

- For a type 1 event, print the rank and score of the team containing participant u.
- For a type 2 event, print the rank and score of the team containing participants u and v after the merge. If they were already in the same team, print the current rank and score of that team.

After all events in the test case, print N more lines. For each participant i from 1 to N, print one line in the format `Rank Solves Penalty` for the final team containing participant i.

Do not print blank lines between test cases.

input

```
1
3 5
1 100 1 1 0
1 150 2 1 1
2 1 2
2 2 1
1 50 1 1 1
```

output

```
1 0 0
1 1 150
1 1 170
1 1 170
1 1 50
1 1 50
1 1 50
2 0 0
```

Initially, participants 1, 2, and 3 are in separate teams.

After the first event, participant 1 has only one wrong answer on problem 1, so the team has score $(0, 0)$.

After the second event, participant 2 has one accepted submission on problem 1 at timestamp 150, so that team has score $(1, 150)$.

After the third event, the teams containing participants 1 and 2 merge. For problem 1, the wrong answer at 100 is before the accepted submission at 150, so the penalty becomes $150 + 20 \cdot 1 = 170$.

The fourth event is a no-op merge because participants 2 and 1 are already in the same team, but the current rank and score must still be printed.

In the fifth event, participant 1 receives an accepted verdict on problem 1 at timestamp 50. Although this event appears later in the input, timestamp 50 is earlier than the previously seen submissions. Therefore, the earliest accepted timestamp becomes 50, and the wrong answer at 100 is no longer before the first accepted verdict. The penalty becomes 50.

H. Capital Logistics

2 seconds, 256 megabytes

The kingdom consists of n cities forming a tree rooted at the capital (city 1). For each city i ($2 \le i \le n$), its parent is p_i. Each city i, including the capital, stores a_i units of goods that must be collected by the royal transport system.

Each day, the royal transport system operates as follows:

- A convoy can collect goods from a set of cities, including the capital city itself.
- Due to road constraints, the convoy can collect from a city C_i only if no other city on the path from the capital to C_i is selected on the same day.
- From each selected city, exactly 1 unit of goods is collected.

Before transport begins, you may discard up to k units of goods in total from any combination of cities to simplify logistics.

Your task is to determine the minimum number of days required to transport all remaining goods to the capital.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases. Then the description of the test cases follows.

The first line of each test case contains two integers n and k ($1 \le n \le 2 \cdot 10^5$, $0 \le k \le 10^{18}$) — the number of cities and the maximum number of goods that can be discarded.

The second line of each test case contains $n - 1$ integers $p_2, p_3, \ldots, p_n$ ($1 \le p_i < i$), where p_i is the parent of city i.

The third line contains n integers $a_1, a_2, \ldots, a_n$ ($0 \le a_i \le 10^9$) — the amount of goods stored in each city.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer — the minimum number of days required to transport all remaining goods to the capital.

input
3 2 3 1 1 2 4 0 1 1 1 1 0 0 0 3 2 1 1 2 1 2

output
0 1 2

In the first test case, we can discard all goods before transport begins, so the answer is 0.

In the second test case, no goods can be discarded. Since only city 1 contains goods, the convoy needs exactly 1 day.

In the third test case, we can discard 2 units of goods from city 1. Then, on the first day, the convoy collects 1 unit from city 2 and city 3 each, and on the second day, it collects 1 remaining unit from city 3. Thus, the answer is 2.

I. The Man, The Math, The Legend!

1 second, 256 megabytes

You are attending the class of Mr. Chamok, one of the best mathematics teachers in the country.

To make the class more interesting, he introduces a game based on the function

$$f(x) = \frac{a}{bx + c} - d$$

Two students, Rasel and Jewel, are playing against each other.

In this game, a real number x will be chosen **uniformly** at random from the interval $[L, R]$. The result of the game will be as follows:

- If $f(x) > 0$, Rasel wins.
- If $f(x) < 0$, Jewel wins.
- If $f(x) = 0$ or $f(x)$ is undefined, the game ends in a tie.

Between Rasel and Jewel, the **expected winner** of this game is the player with the higher winning probability. If both players have equal winning probabilities, there is no expected winner.

Now, Mr. Chamok has given you the task to determine the expected winner of the game.

Input

The first line of the input contains a single integer t ($1 \le t \le 2 \times 10^5$) — the number of test cases.

Each test case consists of a single line containing six space-separated integers a, b, c, d, L, R ($-10 \le a, b, c, d \le 10$, $-100 \le L \le R \le 100$) — the constants defining the function and the range from where x will be chosen.

Output

For each test case, output a single word in a line — the expected winner of the game. If the expected winner is Rasel, output `"Rasel"` (without the quotes). If the expected winner is Jewel, output `"Jewel"` (without the quotes). If there is no expected winner, output `"None"` (without the quotes).

You can output the answer in any case (upper or lower). For example, the strings `"rasel"`, `"Rasel"`, `"RASEL"`, and `"rAsEl"` will all be considered the same.

input
5 10 1 -2 3 3 6 7 2 3 1 4 9 -3 2 4 0 -5 1 4 -2 10 8 5 5 2 5 1 -1 -2 2

output
Rasel Jewel None None Rasel

In the first test case, the probability that Rasel will win the game is $\frac{7}{9}$. In the second test case, Jewel is guaranteed to win. In the third test case, both players have a $\frac{1}{2}$ probability of winning. In the fourth test case, it is guaranteed that $f(x)$ will be undefined. So, neither Rasel, nor Jewel can win.

J. Resonance Frog

3 seconds, 256 megabytes

A magical frog is trying to cross a pond by jumping across a straight line of n giant lily pads, numbered 1 to n. Each lily pad along the way has its own unique musical properties. When the frog stands on pad i, it can sing a specific frequency note called a Launch Tune (a_i) to prepare for a giant leap, while the pad itself constantly hums a natural frequency note known as its Echo Frequency (b_i).

The frog starts its journey on lily pad 1. From its current pad i, the frog can perform one of three actions. Each action costs exactly 1 jump:

1. The frog hops forward to the immediate next pad, landing on $i + 1$. This is valid as long as $i + 1 \le n$.
2. The frog sings its Launch Tune a_i. It listens backward to find the closest previous pad j (where $1 \le j < i$) that is humming a matching Echo Frequency ($b_j = a_i$). Measuring the distance back to that pad $x = i - j$, the magic echo propels the frog exactly that same distance forward to land on destination pad $m = i + x$. This leap is only valid if such a previous pad j exists and the destination m is within the pond ($m \le n$).
3. The frog begins the exact same leap toward destination m as described above, but chooses to cancel its trajectory mid-air and drop onto an intermediate pad k. This is valid only if pad k lies strictly along the flight path ($i < k < m$) and its Echo Frequency matches the frog's active Launch Tune ($b_k = a_i$). The frog can successfully perform this drop even if the original theoretical destination m lies outside the pond ($m > n$), as long as the actual landing pad k is safely within bounds ($k \le n$).

Find the minimum number of jumps required for the frog to reach lily pad n. It is mathematically guaranteed that the frog can always reach the end of the pond.

Input
The first line contains a single integer t ($1 \le t \le 10^4$) denoting the number of test cases.

For each test case:

- The first line contains a single integer n ($1 \le n \le 10^6$) denoting the number of lily pads.
- The second line contains n space-separated integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) representing the Launch Tunes of each pad.
- The third line contains n space-separated integers $b_1, b_2, \ldots, b_n$ ($1 \le b_i \le 10^9$) representing the Echo Frequencies of each pad.

It is guaranteed that the sum of n over all test cases does not exceed 10^6.

Output
For each test case, output a single integer on a new line: the minimum number of jumps required to reach lily pad n.

input
2 10 100 50 50 20 99 100 98 100 97 999 100 20 30 40 50 100 70 100 90 10 3 1 2 3 1 1 1
output
6 2

For the first test case:

- Perform *action 1* consecutively from pad 1 to 4 (requires 3 jumps).
- At pad 4, the Launch Tune is $a_4 = 20$. The closest previous pad humming a matching Echo Frequency is pad 2 ($b_2 = 20$). The distance back to the echo is $x = 4 - 2 = 2$. The frog performs *action 2*, landing exactly 2 steps forward at destination $m = 4 + 2 = 6$.
- At pad 6, the Launch Tune is $a_6 = 100$. The closest previous pad humming a matching Echo Frequency is pad 1 ($b_1 = 100$). The distance back is $x = 6 - 1 = 5$. The magical echo tries to throw the frog forward to destination $m = 6 + 5 = 11$, which is completely out of the pond. However, the frog can perform *action 3* to land early on pad $k = 8$, because it lies strictly under the flight path ($6 < 8 < 11$) and is humming the matching Echo Frequency ($b_8 = 100$).
- Finally, at pad 8, the Launch Tune is $a_8 = 100$. The closest previous pad humming a matching Echo Frequency is pad 6 ($b_6 = 100$). The distance back is $x = 8 - 6 = 2$. The frog performs *action 2*, landing exactly 2 steps forward at destination $m = 8 + 2 = 10$.
- Total jumps required: $3 + 1 + 1 + 1 = 6$.

For the second test case, there are no valid matching frequencies between the frog's Launch Tunes and previous Echo Frequencies. The frog must perform *action 1* consecutively 2 times to reach pad 3.

K. Diagonal Shortcuts

1 second, 256 megabytes

Upobir is playing a game on an infinite 2D grid. He starts at the origin $(0, 0)$ and wants to travel to the target coordinate (x, x). To navigate the grid, Upobir can perform two types of moves:

- **Chess Knight Move:** From his current position (x_1, y_1), he can move to any coordinate (x_2, y_2) such that $|x_1 - x_2| = 1$ and $|y_1 - y_2| = 2$, or $|x_1 - x_2| = 2$ and $|y_1 - y_2| = 1$. This move costs 1 unit of energy.
- **Special Jump:** He can perform a unique jump to instantly teleport from his current position directly to the target (x, x). This unique move does not follow standard chess rules and costs k units of energy.

Given x and k, find the minimum total energy Upobir needs to spend to reach (x, x).

Input
The first line contains a single integer t ($1 \le t \le 10^5$) — the number of test cases.

The only line of each test case contains two integers x and k ($1 \le x \le 10^9$, $1 \le k \le 10^9$) — the coordinate value defining the target (x, x), and the energy cost of the special jump.

Output
For each test case, print a single integer — the minimum total energy required to reach (x, x).

input
3 1 5 3 2 3 3
output
2 2 2

In the first test case, Upobir can reach $(1, 1)$ using 2 chess knight moves: $(0, 0) \to (-1, 2) \to (1, 1)$. This costs 2 units of energy.

In the second test case, Upobir can reach $(3, 3)$ using 2 chess knight moves: $(0, 0) \to (1, 2) \to (3, 3)$. Alternatively, he could use the special jump immediately at $(0, 0)$ to instantly leap to $(3, 3)$. In both scenarios, the minimum energy required is 2.

L. Onion Peeling

15 seconds, 256 megabytes

You are given a set of N points on a 2D plane.

Onion Peeling is the process of repeatedly computing the Convex Hull of the remaining points, then removing those hull points. Each complete set of hull points removed in one iteration is called a **layer**. Let the convex hulls of the onion layers be $H_1, H_2, \ldots, H_L$, where H_1 is the outermost layer (convex hull of all N points) and H_L is the innermost layer.

You are given Q query points. For each query point P, determine how many onion layers strictly contain it. A point is strictly inside a convex polygon if it is in the interior, not on the boundary.

Specifically:

- If P is strictly outside H_1, output 0.
- If P is strictly inside exactly k hulls (H_1 through H_k), output k.
- The maximum possible answer is L (if P is strictly inside all L hulls).

A hull with 1 or 2 vertices has no interior — no point can be strictly inside it.

Input

The first line contains an integer T ($1 \le T \le 9$), the number of test cases. The description of T test cases follows.

Each test case is described as follows:

The first line contains two integers N ($1 \le N \le 5 \times 10^4$) and Q ($1 \le Q \le 10^6$), where N is the number of points and Q is the number of queries. The sum of N over all test cases in a file does not exceed 5×10^4, and the sum of Q does not exceed 10^6.

The next N lines each contain two integers x_i, y_i ($-10^9 \le x_i, y_i \le 10^9$) — coordinates of the i-th point. All N points are distinct and no three points are collinear.

The next Q lines each contain two integers x_j, y_j ($-10^9 \le x_j, y_j \le 10^9$) — coordinates of the j-th query point.

Output

For each test case, output Q lines. The j-th line contains a single integer — the number of layers strictly containing the j-th query point.

input
1 12 5 0 0 12 1 12 12 1 12 8 3 10 5 5 7 8 10 9 5 9 6 3 9 5 4 0 0 6 6 -1 -1 13 6 12 12
output
0 2 0 0 0

Codeforces (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform